MODULE 07 · DOMAIN 4 - TERRAFORM CONFIGURATION

Resources, Data Sources, References, Addressing

The two block types that touch infrastructure, and the syntax that wires them together.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 7 of 27

admin@sthithapragna.com

A **resource** block says Terraform owns this.
A **data** block says somebody else does,
and I need to look at it.

The resource block

- **resource**, then two labels: the **resource type**, then a **name** you choose.
- The type comes from a provider. Its first word is how Terraform picks the plugin, as in module two.
- The name is **local to the module** and has no meaning to the provider. It appears nowhere in your cloud account.
- Together they form the **resource address**: the type, a dot, then the name.
- Two resources in one module cannot share a type and name. That pair must be unique.

> ON THE EXAM
>
> The name is for you and for the dependency graph. Providers never see it, so renaming a resource means Terraform thinks you deleted one and made another.

Arguments or attributes?

ARGUMENTS

What you write in

- Values you supply in the block
- Required or optional, per the provider
- Known before anything is created
- Changing one may update or replace

ATTRIBUTES

What comes back out

- Values the provider reports
- Include everything you supplied, plus more
- Computed ones are unknown until apply
- Referenced by other resources

THE TIEBREAKER

(known after apply) in a plan means a computed attribute. The provider decides it, so Terraform genuinely cannot say in advance.

The four things Terraform can do to a resource

ACTION	WHEN TERRAFORM CHOOSES IT
Create	It is in configuration and has no binding in state
Update in place	An argument changed, and the provider can change it live
Replace	An argument changed that the provider cannot change live
Destroy	It has a binding in state and is no longer in configuration
No action	Configuration, state and reality all agree

ON THE EXAM

The provider decides which attributes force replacement, not Terraform. That is why the same change is trivial on one resource type and destructive on another.

The data block

- **data**, then the data source type, then a name you choose. Same shape as a resource.
- **The address is different.** It carries a data prefix: data, the type, then the name.
- Forgetting that prefix is one of the most common syntax errors in the language.
- Data sources are provided by the same providers, and are declared in the registry alongside resources.
- **A data source creates nothing and destroys nothing.** It appears in state only as a cached read.

> ON THE EXAM
>
> Resource address: type dot name. Data source address: data dot type dot name. That extra segment is examinable.

resource or data?

USE A RESOURCE WHEN

Terraform owns the object

- You want it created and managed
- Its lifecycle is yours to control
- Destroying the config destroys the object
- It appears in state as a binding

USE A DATA WHEN

Something else owns it

- Created by another team or another config
- Created outside Terraform entirely
- You need its attributes, not control of it
- Destroying the config leaves it untouched

THE TIEBREAKER

Ask what should happen on destroy. If the answer is 'that object must survive', it is a data source.

When data sources are read

- **Terraform tries to read data sources during the plan phase**, so their values are available in the plan.
- It **defers the read until apply** when an argument refers to something it cannot predict.
- That happens when the data block depends on a managed resource **configured to change in the current plan**.
- It also happens when a **custom condition** on the block depends on such a resource. Module twelve covers those.
- Terraform **says so in the plan**: 'will be read during apply', with the reason in brackets.

> ON THE EXAM
>
> A deferred read is contagious. Anything referencing that data source becomes unknown at plan time too.

depends_on on a data source, and why to avoid it

- Adding **depends_on** to a data block forces Terraform to wait for the named object before reading.
- If that dependency has **changes pending**, the read is pushed to apply and its attributes show as known after apply.
- **Every resource consuming that data source becomes partially unknown** in the plan too.
- So the plan becomes far less informative, which defeats the reason plans exist.
- **Prefer an implicit dependency** through an attribute reference. Reserve depends_on for what references cannot express.

> ON THE EXAM
>
> depends_on guarantees ordering. It never guarantees the data source will find what it is looking for.

References create dependencies

- Writing one resource's attribute inside another creates an **implicit dependency**.
- Terraform builds the graph from those references, and derives creation order from the graph.
- **This is the mechanism**. It is why module one said Terraform never needs you to write the order of operations.
- Hardcode an identifier as a literal string and **no dependency exists**. Terraform may build them in any order.
- Destroy runs the graph in reverse, using the dependency metadata state retains, as in module four.

> ON THE EXAM
>
> A reference is not just a way to read a value. It is how you tell Terraform that one thing must exist before another.

The reference syntax

TO REFER TO	YOU WRITE
A resource attribute	aws_instance.web.id
A data source attribute	data.aws_ami.ubuntu.id
An input variable	var.region
A local value	local.common_tags
A child module output	module.network.vpc_id
The current workspace name	terraform.workspace

> ON THE EXAM
>
> Six shapes cover almost everything. Only data sources and module outputs need a prefix segment.

Addressing an instance, and a module

- With **count**, instances are addressed by number in square brackets, starting at zero.
- With **for_each**, instances are addressed by their key in square brackets, as a string.
- A **resource address** names the resource as a whole. A **resource instance address** names one instance.
- Inside a module, prefix with **module** and the module name, and repeat for nesting.
- The same syntax is used by **-target**, by the state commands, and by moved and import blocks.

> ON THE EXAM
>
> Learn this syntax once. It reappears in -target, terraform state mv, and the moved, removed and import blocks.

Values that are not resources

- **path.module** is the filesystem path of the module where the expression is written.
- **path.root** is the path of the root module. **path.cwd** is the current working directory.
- Use path.module when a module reads a file it ships with, so it works wherever the module is used.
- **terraform.workspace** gives the current workspace name, which lets configuration vary by environment.
- These are **named values**, not resources. They have no attributes and create no dependencies.

> ON THE EXAM
>
> path.module and path.root differ the moment your code lives in a child module. That difference is the whole point.

Cycles, and what they mean

- If A references B and B references A, Terraform reports a **cycle** and refuses to plan.
- The graph must be acyclic. There is no ordering that satisfies a loop, so Terraform cannot proceed.
- Usually the fix is to notice that **one direction is not really needed**, and remove that reference.
- Sometimes the answer is a **separate association resource** that references both, rather than each other.
- **depends_on can create cycles too**, and is a common cause when it is added carelessly.

> ON THE EXAM
>
> A cycle error is a modelling problem, not a syntax problem. Terraform is telling you the relationship you described is impossible.

Getting a value from somewhere else

A DATA SOURCE WHEN

The platform can tell you

- Query the provider by name or tag
- Always current at plan time
- No coupling to another state file
- Works for anything Terraform did not create

TERRAFORM_REMOTE_STATE WHEN

Another configuration knows

- Reads another configuration's outputs
- Only exposes declared outputs
- Requires access to that backend
- Couples the two configurations together

THE TIEBREAKER

Prefer a data source. Remote state couples two configurations and exposes whatever that state contains to whoever can read it.

A scope note on provisioners

- **Provisioners** run scripts on a local or remote machine as part of create or destroy.
- HashiCorp documents them under the heading **provisioners are a last resort**, and means it.
- They are **invisible to the plan**. Terraform cannot predict or show what a script will do.
- Better answers: **cloud-init or user data** for bootstrapping, and a configuration management tool afterwards.
- **terraform_data** is the built-in replacement for the old null_resource pattern.

> ON THE EXAM
>
> Provisioners are not named in the 004 objectives. Know what they are, know they are discouraged, and do not expect them to be the answer.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
The object must survive when we destroy this config	A data source, not a resource
Resources were created in the wrong order	A hardcoded value where a reference belongs
Plan is full of 'known after apply' unexpectedly	A deferred data source read
Another team owns the network we deploy into	A data source first, remote state second
A module reads a file it ships with	path.module, never path.root
Terraform reports a cycle	Two resources referencing each other. Remove one direction

ON THE EXAM

Ownership decides resource against data. References decide ordering. Those two questions settle most of this module.

A team deploys application servers into a subnet created by the networking team in a separate configuration. To avoid coupling, an engineer looks up the subnet identifier in the console and writes it into the configuration as a literal string.

The deployment works for months. Then a new region is added, and in that region Terraform occasionally attempts to create a security group rule before the security group it belongs to exists, and the apply fails intermittently.

What is the underlying cause?

A Terraform's parallelism is too high and should be reduced with -parallelism=1

B Hardcoded identifiers create no references, so no dependency exists and ordering is undefined

C Security group rules must always be declared before security groups in the file

D The subnet data is stale and the configuration should be re-initialised

◆ PAUSE AND DECIDE

What is the underlying cause?

A Reduce parallelism

Would make the failure rarer by serialising work, and the order would still be arbitrary. It hides an undefined ordering rather than defining it.

B No reference, no dependency

Correct. Terraform derives ordering from references. A literal string carries the right value and declares no relationship at all.

C Declaration order matters

Terraform is declarative. Position in the file has no effect on ordering; only the dependency graph does.

D Stale data, re-init

Init handles backend, providers and modules. It has nothing to do with resource ordering, and no data source is involved here.

THE SIGNAL

It worked for months because Terraform happened to choose a workable order. Nothing changed except the luck.

> A configuration manages an application stack. It must deploy into a shared VPC that predates Terraform and is managed by a central platform team, who tag it with a known name.
>
> A junior engineer writes a resource block for the VPC so that the configuration is complete and self-documenting, then runs plan.

What will happen, and what should be done?

A Terraform will adopt the existing VPC automatically, since the tag matches

B Terraform will plan to create a second VPC. Use a data source to look up the existing one

C Terraform will error, because a resource cannot be declared for an object that already exists

D Terraform will plan correctly, but the VPC will be destroyed on the next apply

◆ PAUSE AND DECIDE

What will happen, and what should be done?

A Adopts it automatically

Terraform never adopts existing objects by matching attributes. Module four explained why tags were rejected as an identity mechanism.

B Creates a second one

Correct. Nothing is in state for that resource, so Terraform plans a create. The platform team owns the VPC, so it is a data source.

C Terraform errors

Terraform has no idea the object exists. It compares configuration with state, and state is empty here, so a create is the honest answer.

D Destroyed next apply

Mixes up the failure. The danger with a wrongly declared resource is destroying somebody else's object later, and that requires importing it first.

THE SIGNAL

Apply the destroy test. If this configuration were destroyed, should the shared VPC go with it? Obviously not. So it is a data source.

A configuration creates a cluster, then uses a data source to read the cluster's generated endpoint, which several downstream resources consume. An engineer adds depends_on to the data block, reasoning that the cluster must exist before it can be read.

The next plan shows the data source as 'will be read during apply', and every downstream resource now shows its arguments as known after apply. Reviewers say the plan has become useless.

What should the engineer do?

A. Keep depends_on and accept it, since ordering correctness matters more than plan readability

B. Remove depends_on and reference a cluster attribute in the data block, creating an implicit dependency

C. Add depends_on to the downstream resources as well, so ordering is explicit throughout

D. Run plan with -refresh=false so the data source is read from cache instead

◆ PAUSE AND DECIDE

What should the engineer do?

A Accept the trade

Presents a false choice. An implicit dependency gives the same ordering guarantee without deferring the read, so nothing has to be traded.

B Reference an attribute instead

Correct. A reference establishes ordering and keeps the value predictable, which is HashiCorp's stated preference over depends_on.

C depends_on everywhere

Spreads the cause rather than removing it. More explicit dependencies on changing resources means more of the plan becoming unknown.

D Plan with -refresh=false

Refresh concerns re-reading existing state. It does not make a deferred data source resolve, and it would hide external changes as well.

THE SIGNAL

depends_on with a changing dependency defers the read. Everything downstream inherits that unknown, which is why the plan emptied out.

A shared module ships a startup script alongside its Terraform files and reads it when configuring instances. The module author tests it from the module's own directory and everything works.

A consuming team adds the module to their root configuration and the apply fails, reporting that the script file cannot be found.

What is the most likely cause?

A The module reads the file using path.root, which resolves to the caller's directory

B Shared modules cannot include files other than .tf files

C The consuming team must run terraform get before the file becomes available

D The file must be passed in as an input variable; modules cannot read files at all

◆ PAUSE AND DECIDE

What is the most likely cause?

A path.root instead of path.module

Correct. In the author's own directory the two are identical, so the bug is invisible until somebody calls the module from elsewhere.

B Only .tf files allowed

Modules routinely ship scripts, templates and policy documents. Nothing restricts a module directory to Terraform files.

C Run terraform get

terraform get installs modules, and init already does it. The module was clearly installed, since Terraform reached the point of reading the file.

D Pass the file in

Modules can read files perfectly well. Passing the content in would work and would push a detail onto every consumer for no reason.

THE SIGNAL

Testing a module from its own directory cannot catch this class of bug, because that is the one place both paths agree.

Module 7 recap

- **Resource means Terraform owns it. Data means somebody else does.**
- **Data source addresses carry a data prefix.** Three segments, not two.
- **References create dependencies.** A hardcoded identifier gives no ordering.
- **Data reads defer to apply** when they depend on something changing.
- **depends_on on a data block is usually wrong.** Prefer an attribute reference.
- **path.module for anything a module ships.** path.root breaks elsewhere.

> ON THE EXAM
>
> Arguments go in, attributes come out, and computed attributes are why plans say 'known after apply'.